3주차

클로드 코드 시작하기

설치부터 손글씨 인식 프로그램, 그리고 CLAUDE.md까지

신구대학교 컴퓨터소프트웨어학과 · 앱프로그래밍응용 (2026-1)

오늘 배울 내용

이 수업의 3대 핵심 역량

역량	설명
바이브 코딩	AI 코딩 파트너를 활용, 자연어 프롬프트로 빠르게 개발
컴퓨팅 사고	문제 분해 → 패턴 인식 → 추상화 → 알고리즘적 해결
프로젝트 관리	요구사항 → 일정 계획 → 중간 보고 → 최종 발표

오늘의 학습 목표

1. **클로드 코드를 설치**하고 터미널에서 첫 대화를 실행할 수 있다.
2. **클로드 코드로 MNIST 손글씨 인식 프로그램**을 만들 수 있다.
3. **CLAUDE.md 파일을 활용**하여 프로젝트를 체계적으로 관리하고 프로그램을 확장할 수 있다.

> 📖 *교재: 혼자 공부하는 바이브 코딩 with 클로드 코드 — Chapter 03 (p.84~125)*

지난 주 복습

2주차에서 배운 것

- **5W1H 프롬프트** — Why, Who, What, When, Where, How로 체계적 질문
- **PRD (제품 기획서)** — 좋은 프롬프트 = 잘 만든 기획서
- **4단계 포트폴리오 전략** — 뼈대 → 기능 → 디자인 → 점검

이번 주 핵심 질문

> 💬 *"웹 브라우저가 아닌 **내 컴퓨터의 터미널**에서 AI와 대화하면 무엇이 달라질까?"*

> 2주차까지는 **클로드 웹(claude.ai)**에서 Artifacts로 작업했습니다. 이번 주부터는 **클로드 코드 (Claude Code)**로 내 컴퓨터에서 직접 코드를 생성하고 실행합니다!

이번 주의 컴퓨팅 사고

컴퓨팅 사고의 **4가지 사고력**이 이번 주에 어떻게 적용되는지 살펴봅시다.

사고력	의미	이번 주 적용
분해	큰 문제를 작은 단위로 나누기	설치 → 프로그램 제작 → 확장의 3단계로 분해
패턴 인식	반복되는 구조 찾기	프롬프트 → 코드 생성 → 실행 → 확인 사이클의 반복
추상화	핵심만 뽑아 정리하기	CLAUDE.md로 프로젝트 규칙을 추상화
알고리즘	순서대로 절차 밟기	환경 준비 → 설치 → 실행 → 확장의 단계적 절차

💡 CLAUDE.md는 프로젝트의 **"설명서"**를 만드는 것 = **추상화**의 실전 적용!

3대 역량, 이번 주 적용

역량	이번 주 활동
바이브 코딩	터미널에서 클로드 코드로 AI와 대화하며 프로그래밍
컴퓨팅 사고	문제 분해(설치→제작→확장) + CLAUDE.md 추상화
프로젝트 관리	CLAUDE.md로 프로젝트 구조와 규칙을 문서화

바이브 코딩 = **터미널에서** AI와 대화 (도구)
컴퓨팅 사고 = 문제를 **단계별로 분해**하여 해결 (사고)
프로젝트 관리 = **CLAUDE.md**로 프로젝트 체계화 (계획)

클로드 코드란?

클로드 코드(Claude Code) 는 터미널(명령 프롬프트)에서 실행되는 AI 코딩 도구입니다.

클로드 웹 vs 클로드 코드

구분	클로드 웹 (claude.ai)	클로드 코드 (Claude Code)
실행 환경	웹 브라우저	내 컴퓨터의 터미널
작업 방식	Artifacts에서 미리보기	실제 파일 생성·실행
활용 범위	간단한 코드, 문서, 웹페이지	복잡한 프로그램, 프로젝트 관리
파일 접근	제한적	내 컴퓨터 파일에 직접 접근

💡 클로드 코드는 내 컴퓨터에서 직접 코드를 만들고 실행하므로, **실제 프로그램을 개발**할 수 있습니다!

터미널이란?

터미널(Terminal) 은 텍스트 명령어로 컴퓨터를 조작하는 도구입니다.

Windows에서 터미널 실행

1. 윈도우 **작업 표시줄의 검색창**에 `cmd` 입력 → Enter
2. 또는 `Windows 키 + R` → `cmd` 입력 → 확인

기본 터미널 명령어

명령어	기능	예시
`dir`	현재 폴더의 파일 목록	`dir`
`cd`	폴더 이동	`cd VibeCoding`
`mkdir`	새 폴더 만들기	`mkdir Study-01`

클로드 코드 설치를 위한 준비물

클로드 코드를 설치하기 전에 **2가지 프로그램**을 먼저 설치해야 합니다.

1. Node.js 설치

- 공식 사이트: https://nodejs.org
- **LTS(Long Term Support)** 버전 다운로드
- 설치 후 확인: 터미널에서 `node --version`

2. Git 설치

- 공식 사이트: https://git-scm.com
- 기본 설정으로 설치 진행
- 설치 후 확인: 터미널에서 `git --version`

✏️ 셀프 체크 — 03-1 클로드 코드 설치하기

아래 문제를 스스로 풀어 보세요 (교재 p.98~99)

문제 1. 클로드 코드를 실행하기 위해 필수적으로 설치해야 하는 두 가지 프로그램을 고르세요.

① Python과 Visual Studio Code
② Node.js와 Git
③ Docker와 Kubernetes
④ Chrome과 Firefox

문제 2. 윈도우 운영체제에서 터미널을 실행하는 명령어는 무엇일까요?

① terminal
② explorer
③ cmd
④ console

✏️ 셀프 체크 — 03-1 (계속)

문제 3. `npm install -g @anthropic-ai/claude-code`에서 `-g` 옵션의 의미는?

① Git과 연동하여 설치
② 그래픽 모드로 설치
③ 시스템 전체에서 사용 가능하도록 설치
④ 게스트 모드로 설치

실습을 시작합니다!

지금부터 **클로드 코드를 설치**하고, **손글씨 인식 프로그램**을 만들고, **CLAUDE.md로 확장**합니다.

실습 흐름

```
① 환경 준비      →  ② 클로드 코드 설치  →  ③ 손글씨 인식 프로그램  →
④ CLAUDE.md 생성 →  ⑤ 웹 버전 확장     →  ⑥ 결과 확인
```

준비물

- 인터넷 연결된 PC (Windows)
- 명령 프롬프트 (cmd)
- Claude 계정 (https://claude.ai) 또는 API 키
- Node.js, Git 설치 완료

실습 ① 클로드 코드 설치하기

터미널에서 클로드 코드를 설치하고 첫 대화를 나눕니다.

설치 명령어

```
npm install -g @anthropic-ai/claude-code
```

실행

```
claude
```

초기 설정 순서

1. **테마 선택** — 밝은 배경 / 어두운 배경 중 선택
2. **로그인** — Claude 계정 또는 API 키로 인증
3. **보안 확인** — 파일 접근 권한에 대한 안내 확인

실습 ① 클로드 코드 첫 대화

설치와 로그인이 완료되면, 터미널에서 바로 AI와 대화할 수 있습니다.

첫 대화 예시 (교재 p.96)

```
> 안녕! 네가 어떤 기능을 가지고 있는지 설명해 줘.
```

클로드 코드가 자신의 다양한 기능들을 친절하게 안내해 줍니다.

> 💡 클로드 코드를 종료하려면 `Ctrl + C`를 **두 번 연속**으로 누르세요.

실습 ① 설치 및 첫 대화 확인

[확인] 체크리스트

- [ ] `npm install -g @anthropic-ai/claude-code` 설치 성공
- [ ] `claude` 명령으로 클로드 코드 실행 성공
- [ ] 테마 설정 및 로그인 완료
- [ ] 첫 번째 대화에 정상 응답 확인
- [ ] `Ctrl + C` 두 번으로 종료 확인

> 축하합니다! 이제 여러분의 컴퓨터에 **AI 코딩 파트너**가 설치되었습니다. 🎉

실습 ② 손글씨 인식 — MNIST란?

손글씨 인식(OCR) 프로그램을 클로드 코드로 만들어 봅시다.

OCR (Optical Character Recognition)

이미지 속 글자를 컴퓨터가 읽을 수 있는 디지털 텍스트로 변환하는 기술

MNIST 데이터셋

MNIST = **M**odified **N**ational **I**nstitute of **S**tandards and **T**echnology

항목	내용
용도	숫자(0~9) 손글씨 인식 학습에 사용
데이터	28×28 픽셀 흑백 이미지 **7만 장**
특징	딥러닝 입문자들이 가장 먼저 접하는 표준 데이터셋

실습 ② 프로젝트 폴더 준비하기

클로드 코드를 사용하기 전에 **프로젝트 폴더**를 먼저 만듭니다.

방법 1 — 터미널 명령어로 만들기

```
mkdir VibeCoding
cd VibeCoding
mkdir Study-01
cd Study-01
```

방법 2 — 탐색기에서 바로 터미널 열기 (교재 p.101~102)

1. **탐색기**에서 원하는 폴더로 이동
2. **주소창**을 클릭하고 `cmd` 입력 → Enter
3. 해당 폴더에서 바로 터미널이 열립니다!

실습 ② 프로젝트 폴더에서 클로드 코드 시작

프로젝트 폴더에서 아래 명령어로 클로드 코드를 실행합니다.

```
claude
```

> 클로드 코드는 실행된 폴더를 **"작업 공간"**으로 인식합니다. 항상 프로젝트 폴더에서 실행하세요!

실습 ② 프롬프트로 프로그램 만들기

클로드 코드에게 **손글씨 인식 프로그램**을 요청합니다.

프롬프트 입력

```
손글씨로 숫자를 입력하면 이것을 인식하는 코드를 만들어서
실행해 줘. 모든 코드와 주석을 영어로 작성해 줘.
```

클로드 코드가 하는 일

1. `digit_recognition.py` — 메인 프로그램 코드 생성
2. `requirements.txt` — 필요한 패키지 목록 생성
3. 자동으로 패키지 설치 및 프로그램 실행

> 💡 클로드 코드는 "코드 작성 → 패키지 설치 → 실행"까지 **한 번에** 처리합니다!

실습 ② 프로그램 실행 결과

클로드 코드가 만든 프로그램을 확인합니다.

실행 결과

- **손글씨 그리기 창** — 마우스로 숫자를 그릴 수 있는 캔버스
- **Recognize 버튼** — 클릭하면 AI가 숫자를 인식
- **신뢰도 표시** — 인식 결과와 정확도(%)를 함께 출력

[확인] 체크리스트

- [ ] 손글씨 그리기 창이 정상적으로 표시되는가?
- [ ] 숫자를 그린 후 Recognize 버튼이 동작하는가?
- [ ] 인식 결과와 신뢰도가 올바르게 출력되는가?

실습 ② 배치 파일로 독립 실행

매번 클로드 코드를 열지 않아도 **더블클릭 한 번**으로 프로그램을 실행할 수 있습니다.

클로드 코드에게 요청 (교재 p.108)

```
숫자 인식 프로그램을 윈도우 탐색기에서 클릭해서
실행할 수 있게 해 줘.
```

결과: `.bat` 파일 생성

배치 파일(.bat)을 더블클릭하면 프로그램이 바로 실행됩니다.

> 💡 **배치 파일(Batch File)**은 Windows에서 여러 명령어를 한 번에 실행하는 스크립트입니다. 확장자는 `.bat`

✏️ 셀프 체크 — 03-2 손글씨 인식 프로그램

(교재 p.111~112)

문제 1. 클로드 코드를 실행할 작업 폴더로 이동하는 방법이 **아닌 것**을 고르세요.

① 탐색기 주소창에 cmd를 입력하여 해당 폴더에서 터미널 실행
② 터미널을 실행한 뒤 cd 명령어로 원하는 폴더로 이동
③ 클로드 코드가 자동으로 최적 폴더를 선택
④ 원하는 폴더에서 마우스 오른쪽 버튼을 클릭해서 터미널 열기

문제 2. 손글씨 인식 프로그램 학습 과정에서 관찰할 수 있는 지표로 옳은 설명을 고르세요.

① accuracy는 낮을수록 성능이 좋다.
② loss는 높을수록 성능이 좋다.
③ accuracy는 높을수록, loss는 낮을수록 성능이 좋다.
④ 두 지표 모두 높을수록 성능이 좋다.

✏️ 셀프 체크 — 03-2 (계속)

문제 3. 클로드 코드의 권한 설정 옵션이 **아닌 것**을 고르세요.

① Ask every time (매번 확인)
② Yes, for this session only (현재 세션만)
③ Always allow in this project (프로젝트 내 항상 허용)
④ Never allow modifications (수정 절대 불가)

실습 ③ CLAUDE.md란?

CLAUDE.md는 프로젝트의 구조, 규칙, 명령어를 정리한 **설정 파일**입니다.

쉽게 이해하기

비유	설명
사용 설명서	프로젝트에 대한 모든 정보를 담은 문서
프로젝트 지도	폴더 구조와 역할을 한눈에 파악
팀 규칙 문서	AI가 따라야 할 코딩 규칙 정의

실습 ③ CLAUDE.md의 구조 (교재 p.116)

항목	내용
Project Overview	프로젝트 목적과 소개
Commands	자주 쓰는 명령어 모음
Tech Stack	사용 기술/라이브러리
Architecture	폴더 구조와 역할
Code Style	코딩 규칙과 스타일
Development Notes	개발 참고 사항

> CLAUDE.md = 클로드 코드에게 주는 **"프로젝트 매뉴얼"**

실습 ③ /init으로 CLAUDE.md 생성하기

클로드 코드의 `/init` 명령어로 **자동으로 CLAUDE.md를 생성**할 수 있습니다.

```
/init
```

/init이 하는 일

1. 프로젝트 폴더의 파일과 구조를 **자동 분석**
2. 분석 결과를 바탕으로 **CLAUDE.md 파일 생성**
3. 프로젝트 목적, 구조, 규칙을 문서화

[확인] 체크리스트

- [ ] `/init` 명령어가 정상 실행되었는가?
- [ ] CLAUDE.md 파일이 프로젝트 루트에 생성되었는가?
- [ ] 파일 내용에 프로젝트 구조가 정확히 반영되었는가?

실습 ③ # 키로 규칙 빠르게 추가하기

CLAUDE.md에 **규칙을 빠르게 추가**하는 단축키가 있습니다.

`#` 키 사용법

클로드 코드 대화 중 `#` 키를 누르면 **빠른 규칙 추가 모드**가 활성화됩니다.

예시 (교재 p.117)

```
# 앞으로 만드는 모든 파일은 날짜와 시간을 주석으로 표시해 줘.
```

저장 위치 선택

옵션	저장 위치	범위
User memory	`~/.claude/CLAUDE.md`	모든 프로젝트에 공통 적용
Project memory	`./CLAUDE.md`	현재 프로젝트에만 적용

실습 ③ 계층적 구조로 프로젝트 확장

CLAUDE.md는 **계층적 구조**로 폴더별 설정을 나눌 수 있습니다.

계층적 구조란?

```
VibeCoding/
├── CLAUDE.md                ← 공통 규칙 (전체 프로젝트)
├── desktop_version/
│   ├── CLAUDE.md             ← 데스크톱 버전 전용 규칙
│   └── digit_recognition.py
└── web_version/
    ├── CLAUDE.md            ← 웹 버전 전용 규칙
    └── (웹 관련 파일들)
```

핵심 원리

- **루트 CLAUDE.md** — 프로젝트 전체에 적용되는 공통 규칙
- **하위 폴더 CLAUDE.md** — 해당 폴더에만 적용되는 특화 규칙

실습 ③ 웹 버전 손글씨 인식 만들기

CLAUDE.md 계층 구조를 활용하여 **웹 버전을 추가**합니다.

프롬프트 입력

```
손글씨 인식 프로그램을 웹 버전과 데스크톱 버전으로 나누어
개발하고 싶어. web_version과 desktop_version 폴더를 만들고
각 폴더에 해당하는 CLAUDE.md 파일도 생성해 줘.
```

클로드 코드가 하는 일

1. `desktop_version/` 폴더 — 기존 프로그램 이동
2. `web_version/` 폴더 — 웹 버전 코드 생성
3. 각 폴더에 **전용 CLAUDE.md** 생성
4. 루트에 **공통 CLAUDE.md** 업데이트

실습 ③ 웹 버전 실행 및 확인

폴더 구조가 완성되면 **웹 버전을 실행**합니다.

프롬프트 입력 (교재 p.121)

```
웹 버전 프로그램을 브라우저에서 실행해 줘.
```

실행 방법

웹 서버가 시작되면 `localhost:포트번호`로 접속

웹 브라우저에서 확인

- 브라우저에서 손글씨를 그리고 인식 결과 확인
- 데스크톱 버전과 동일한 기능이 웹에서 동작

실습 ③ 웹 버전 확인 체크리스트

- [ ] `web_version/`, `desktop_version/` 폴더가 생성되었는가?
- [ ] 각 폴더에 CLAUDE.md가 존재하는가?
- [ ] 웹 버전이 브라우저에서 정상 동작하는가?
- [ ] 데스크톱 버전도 여전히 정상 실행되는가?

> **프로젝트 관리 연결**: CLAUDE.md로 프로젝트 구조를 체계화하는 것은 실무에서의 **문서화 습관**과 동일합니다!

실습 중 자주 발생하는 문제

문제	원인	해결 방법
`npm`이 인식되지 않음	Node.js 미설치 또는 PATH 미등록	Node.js LTS 버전 재설치
`git`이 인식되지 않음	Git 미설치	git-scm.com에서 설치
클로드 코드 로그인 실패	계정 인증 오류	Claude 계정 재로그인 시도
손글씨 인식 창이 안 뜸	Python 또는 패키지 미설치	클로드에게 "패키지 다시 설치해 줘" 요청
`/init` 후 CLAUDE.md 없음	폴더 권한 또는 경로 문제	올바른 프로젝트 폴더에서 재실행

핵심 키워드 — 이론 (교재 p.99, 112)

- **터미널(Terminal)** — 텍스트 명령어로 컴퓨터를 조작하는 도구
- **클로드 코드(Claude Code)** — 터미널에서 실행되는 AI 코딩 도구
- **명령 프롬프트(cmd)** — Windows의 기본 터미널 프로그램
- **OCR** — 이미지 속 글자를 디지털 텍스트로 변환하는 기술
- **MNIST** — 0~9 손글씨 숫자 이미지 7만 장의 학습 데이터셋

핵심 키워드 — 실습 (교재 p.112, 125)

- **npm** — Node.js의 패키지 관리자 (Node Package Manager)
- **배치 파일(.bat)** — 여러 명령어를 한 번에 실행하는 Windows 스크립트
- **CLAUDE.md** — 프로젝트의 구조, 규칙, 명령어를 기록해 클로드 코드가 자동으로 읽는 설정 파일
- **/init** — 현재 폴더를 분석해 CLAUDE.md를 자동 생성하는 초기화 명령어
- **# 키** — 대화 중 현재 작업의 맥락이나 규칙을 CLAUDE.md에 빠르게 추가하는 지시어
- **계층적 구조** — 루트에 공통 규칙을 두고, 하위 폴더마다 특화 규칙을 나눠 적용하는 관리 방식

✏️ 셀프 체크 — 03-3 CLAUDE.md

(교재 p.124~125)

문제 1. CLAUDE.md 파일의 주요 역할이 **아닌 것**을 고르세요.

① 프로젝트 맥락을 자동으로 복원
② 코딩 규칙과 명령어 저장
③ 이전 대화 내용 전체 저장
④ 클로드 코드가 시작할 때 자동으로 읽어들임

문제 2. CLAUDE.md 파일을 자동으로 생성하는 명령어를 고르세요.

① /start ② /init ③ /create ④ /setup

✏️ 셀프 체크 — 03-3 (계속)

문제 3. `#` 의 실제 동작 방식으로 올바른 것을 고르세요.

① 입력한 내용을 즉시 CLAUDE.md에 자동 저장
② 저장 위치를 선택하는 프롬프트를 표시한 후 저장
③ 현재 세션에만 임시로 규칙 적용
④ 모든 프로젝트의 CLAUDE.md를 동시에 업데이트

문제 4. 계층적 CLAUDE.md 구조의 장점으로 옳지 **않은 것**을 고르세요.

① 공통 규칙을 루트에 한 번만 작성
② 하위 프로젝트별 특화 규칙 적용 가능
③ 토큰 사용량이 자동으로 감소
④ 전체 프로젝트의 일관성 유지

오늘의 마무리 체크리스트

수업이 끝나기 전에 아래 항목을 확인해 보세요!

- [ ] Node.js와 Git이 정상 설치되어 `node --version`, `git --version` 출력 확인
- [ ] 클로드 코드 설치 및 로그인 완료 (`npm install -g @anthropic-ai/claude-code`)
- [ ] 터미널에서 `claude` 실행 후 첫 대화 성공
- [ ] 손글씨 인식 프로그램 실행 및 숫자 인식 확인
- [ ] `/init`으로 CLAUDE.md 생성 완료
- [ ] "내가 오늘 배운 것" 2줄 메모

> ✅ 손글씨 인식 프로그램 실행 화면 캡처 + CLAUDE.md 파일 내용을 교수님께 제출하면 오늘 수업 완료!

과제 안내

이번 주 과제

오늘 만든 손글씨 인식 프로그램을 **웹 버전으로 확장**하여 완성하세요.

과제 요구사항

- 클로드 코드로 **웹 버전 손글씨 인식 프로그램** 완성
- CLAUDE.md 계층 구조 적용 (루트 + web_version + desktop_version)
- 실행 결과 **스크린샷** 제출

다음 주 예고

> ***4주차**: 클로드 코드 실전 활용 — 더 복잡한 프로그램을 만들고 디버깅하기*

> 오늘 배운 "클로드 코드 설치"와 "CLAUDE.md 활용"은 앞으로 **모든 프로젝트의 출발점**이 됩니다!